

M-18 Netflix Prize

V1: Business/Real world problem:

Problem Definition

$U_i \rightarrow \text{user } i$

$M_j \rightarrow \text{movie } j$

$r_{ij} \rightarrow \text{Rating based on } i, j$
(1-5)

RMSE $\rightarrow$ Metric

V2: Objectives and Constraints

$U_i, M_j, \hat{r}_{ij} \rightarrow \text{predicted}$

$r_{ij} \rightarrow \text{actual}$

Some form of
interpret
ability

Minimize $\sum_{i,j} (r_{ij} - \hat{r}_{ij})^2 \rightarrow \text{This is what RMSE \& MAPE}$

4) But why minimize the rating?

Netflix could precompute in a hashtable or lookup table on a nightly basis. So no need of latency constraints.

V3: Mapping to an ML problem: Data Overview

Download data on kaggle, 4 txt files and 1 csv file.

Data point :-

Movie ID, Rating, watched on date
YYYY:MM:DD

↓
17,770
total

↓
1 to 5

Movie & Watch on date

Each movie ID has a Customer ID, Rating

V4: Mapping to an ML problem: ML problem formulation

1) Recommendation problem → Similarity based
→ Matrix Factorization

2) Regression problem.

↓
V5: Exploratory Data Analysis: Data preprocessing
movie user rating
df = m, u, M, u, r, data

V6: Exploratory Data Analysis: Temporal Train-test Split
Time order

V7: Exploratory Data Analysis: Preliminary Data Analysis day-of-week also added
V8: Exploratory Data Analysis: Sparse Matrix representation

Movie_ID USER_ID RATING
↓
Capital

★ CSR-Matrix uses only 1% of RAM out of all the data.

V9: Exploratory Data Analysis: Average ratings for various slices

I want to compute:-

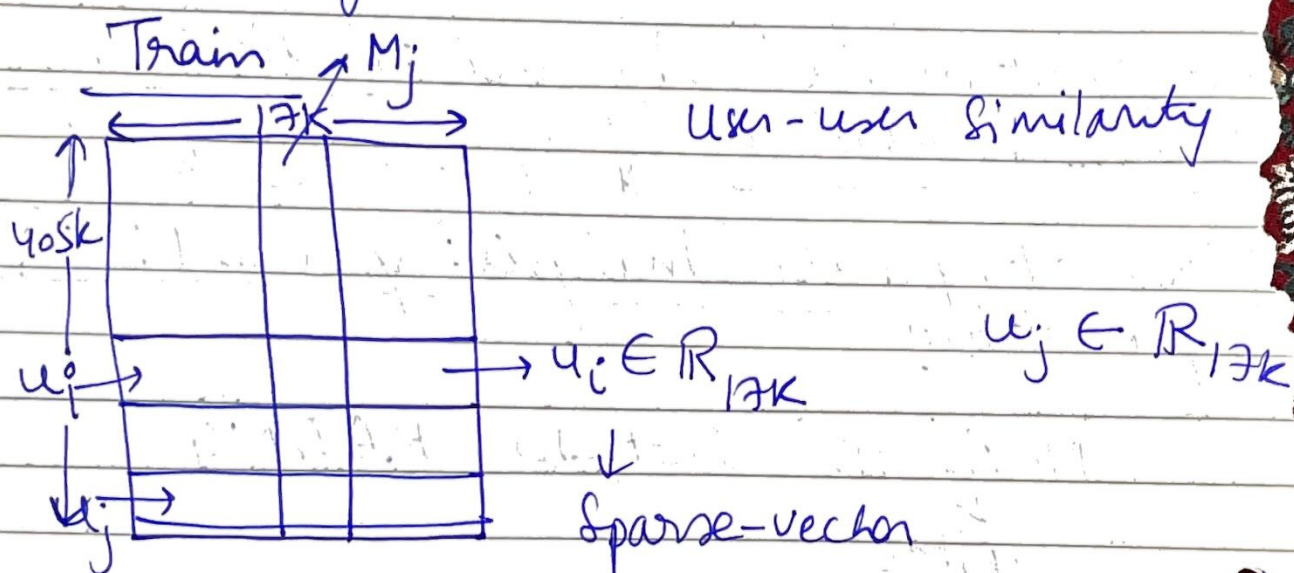
- 1) Global mean of all ratings (μ)
- 2) User averages (u_i) = Vector (critical/lenient)
- 3) Movie averages (m_j) = Vector

Sim

V10: EDA: Cold Start problem

When new data is present in test and no similar data could be derived from train.

V11: Computing Similarity Matrices: User-User Similarity Matrix



$$\text{Cosine}(u_i, u_j) = u_i^T \cdot u_j \rightarrow \text{Dot Product}$$

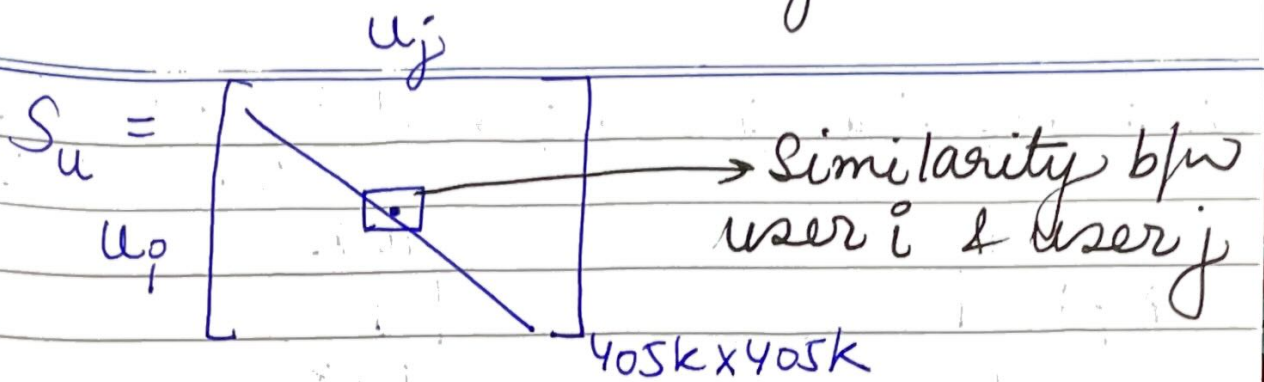
$$\sum_{k=1}^{1 \times K} u_{ik} u_{jk}$$

M_j

$$A = \begin{bmatrix} u_0 \\ u_1 \\ u_j \end{bmatrix}$$

$405K \times 17K$

Similarity matrix is not sparse but is symmetric.



$$(S_u)_{ij} = u_i^T u_j = u_j^T u_i$$

$$S_{ij} = S_{ji}$$

So, $405k \times 405k$

altern suggested conversion of $u_i \in 17k$ to 500 dim u_{ij} using SVD. SVD computes & converts high dim to low dim but high dim is sparse & low dim is dense.

For any user u_i we actually compute it there and then using dictionary of dictionaries.

PCA & SVD returns a dense vector even after you reduce dimension.

We're not using Similarity matrix for this case study.

V12: Computing Similarity matrices: Movie-Movie Similarity

M_i : 405K

M_j : 405K

Extremely sparse.

Similarity Matrix (S_m) = $(17K \times 17K)$ (144MM) Comput
 $= m_i^T m_j \div 2$

Symmetric & dense.

Dictionary of dictionaries

Key: M_i

Value:

Key	Value
All the movies	$M_{i,j}$ / Similarity value

Dictionary of dictionaries	M_1	M_1	Similarity value
	M_2	M_1	"
	M_3	M_1	"

V13: Computing Similarity matrices: Does movie-movie similarity work

Shows related movie.

V14: ML Models: Surprise library

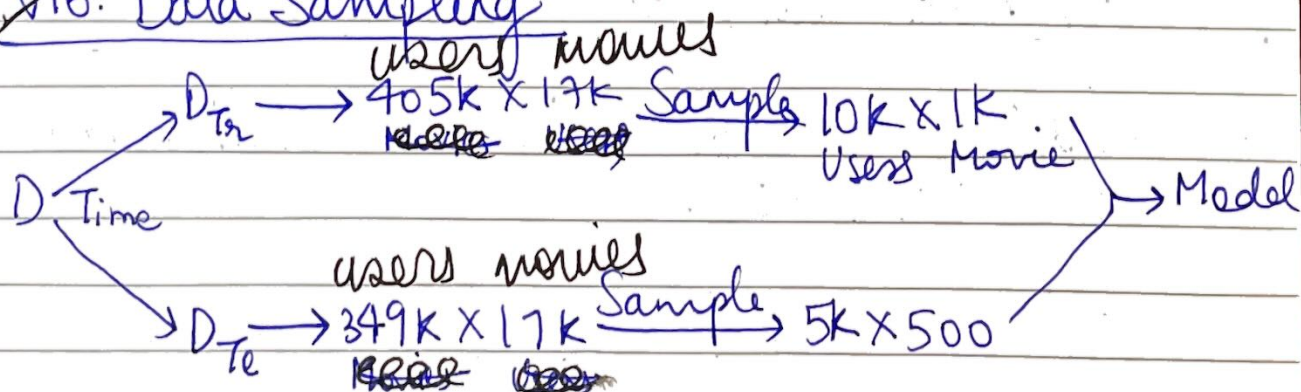
Collection of Yehuda Koren's work in Netflix Competition in a research paper.

SURPRISE: Simple Give data in triplet format U_i, M_j, r_{ij}

V15: Overview of the modelling strategy

- 1) Xgboost 13 feature hand-crafted.
- 2) ~~Xgboost~~ + Surprise Baseline model.
- 3) ~~Xgboost + Surprise~~

V16: Data Sampling



V-17: Google drive with intermediate files check train and test npz files.

V-18: Featurizations for regression

13 handcrafted features.

From sampled data, we'll calculate average.

Features

- 1st - Global average (μ) $\nearrow s_{u0}, s_{u02}, s_{u03}, s_{u05}$
- 2nd - Similar users rating of this movie
- 3rd - Similar movies rated by this user
- 4th - User's Avg rating $s_{m0}, s_{m02}, s_{m04}, s_{m05}$
- 5th - Average rating of this movie
- 6th - rating (y_{ij}) $\nearrow r_{ij} \rightarrow y$

x_{ij} Calculate for both train & test!

x_{ij}

--	--	--

 y_{ij}

--

 $u_i, M_j \leftarrow x_{ij}$

V-19: Data Transformation for Surprise

V-20: Xgboost with 13 features

Metric is stored in a dictionary of dictionaries.

Try to get feature-importances.

RMSE
MAPE - % error

V21: Surprise Baseline model

u : user

i : item(movie)

r_{ui} actual rating

$\hat{r}_{ui}$ predicted rating

b_{ui} = baseline model

μ = ^{Global} average of all the ratings in the training data

b_u : User bias

b_i : Item bias (Movie bias)

$\hat{r}_{ui} = b_{ui} = \mu + b_u + b_i$ (Extremely simple linear model)

Optimization Function

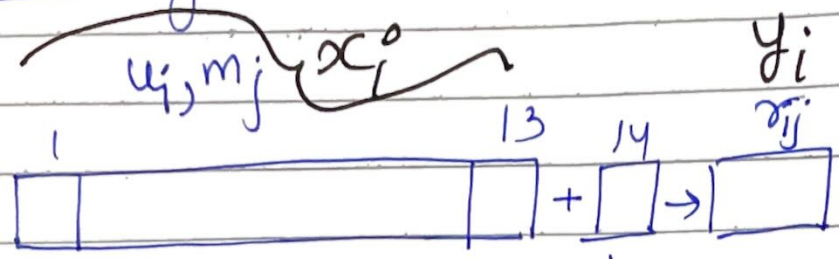
$$\min_{b_u, b_i} (r_{ui} - \hat{r}_{ui})^2$$

$$\sum_{r_{ui} \in R_{\text{train}}} (r_{ui} - (\mu + b_u + b_i))^2 + \lambda (b_u^2 + b_i^2)$$

$r_{ui} \in R_{\text{train}}$

$[\min b_u, b_i]$

✓ V22: Xgboost + 13 features + Surprise baseline model



↓
bs/pr — comes out be
least imp. feature
baseline → predicted

$u_i, m_j \xrightarrow{\text{baseline}} r_{ij}$

KNN $\xrightarrow{\text{KNN predicted}}$ (15) feature
Surprise

F-Score (Feature Score).

✓ V23: Surprise KNN predictors
Surprise baseline

$$\hat{r}_{ui} = b_{ui} + \sum_{v \in N_i^k(u)} \text{sim}(u, v) \cdot (r_{vi} - b_{vi})$$

Set of k similar users (neighbors of user(u) who rated movie i) $\sum_{v \in N_i^k(u)} \text{sim}(u, v)$

open base line model

carson correlation coefficient better than cosine similarity here which is modified to Shrink Pearson Correlation coefficient.

Shrinkage — sort of Laplace smoothing
no. of k
min support } hyperparameters

V24: Matrix Factorization models using Surprise

SVD

V25: SVD++ with implicit feedback

SVDpp

$$\hat{r}_{ui} = \underbrace{\mu + b_u + b_i}_{\text{baseline}} + \underbrace{q_i^T b_u}_{\text{SVD (explicit feedback)}} +$$

$$q_i^T \left(|I_u|^{-1/2} \sum_{j \in I_u} y_j \right)$$

Implicit feedback.

$u, i \rightarrow r_{ij}$ explicit feedback.

after reading the $\rightarrow$ implicit feedback content

V26: Final models with all features and predictors

Xgboost ^{with} 13 features

Surprise baseline model

Surprise kNN

SVD

SVD++.

V27: Comparison between various models

Assignment 18: Netflix Prize

- 1) 25K users } Try all user
3K Movies } & all movies

Report RMSE and MAPE on test data.
Train model on the above code (mentioned in ipynb)

- 2) Tune hyperparameters of all the XGBoost models above to improve RMSE.